

Extended Abstract

Telemetry-Driven Runtime Control for Heterogeneous LLM Agent Systems

Anonymous Author(s)

Abstract

LLM agent systems fail in ways that production tracing stacks cannot see. AGENTSCOPE is an OpenTelemetry SDK that closes the gap with (i) a nine-kind agent-orchestration span vocabulary extending GenAI semantic conventions, (ii) seven framework adapters that solve the cross-process correlation problem, and (iii) a telemetry-driven *circuit breaker* that turns the same span stream into runtime control. Measured on commodity hardware: 11.7 μ s p50 per span, 19,071 spans/s under 100-thread concurrency, +2.16 μ s p50 breaker overhead (95% CI [+2.06, +2.25] across 5 independent runs), and 0.612 (vs 0.429 baseline; 1.000 on a conformance-complete adapter) fault-detection rate over a 3,780-row controlled benchmark spanning 7 frameworks $\times$ 6 LLMs $\times$ 14 fault classes.

Problem

Production LLM agent systems — multi-step planners that compose reasoning, tool calls, memory operations, and inter-agent delegation — have entered the critical path of software engineering, customer support, knowledge work, and incident response. The tracing stack that was built for microservices does not see what they do. An OTel INTERNAL span carries no semantic information distinguishing a guardrail rejection from a reasoning step or a delegation handoff. The in-stabilization OpenTelemetry GenAI semantic conventions name LLM invocations and tool executions but stop short of the orchestration phases that surround them. Closed-source platforms (LangSmith, Langfuse) and adjacent OSS schemas (OpenInference, OpenLLMetry) capture richer attributes per vendor but ship vocabularies that do not interoperate.

The operational consequence is concrete: in a prior controlled study, 84 of 112 SWE-bench Lite runs against a production agent exhausted their iteration budget with *no* span-level evidence of why they failed — no record of which planning step diverged, which tool call repeated, or which delegation cycle never closed.

This is not a documentation problem; it is a systems problem with three dimensions: (D1) heterogeneous instrumentation surfaces across seven popular frameworks each with a different hook model; (D2) span correlation across asyncio / multi-process / inter-agent boundaries; (D3) passive tracing cannot enforce runtime policies (cost, loop, delegation cycle) without a control surface wired into the span stream.

Approach

AGENTSCOPE is an OTel-native SDK addressing D1–D3 with measured trade-offs.

Span vocabulary. Nine agent-specific kinds (AGENT, PLANNING, REASONING, DELEGATION, GUARD_RAIL, MEMORY, RETRIEVAL, TOOL_CALL, LLM_CALL) implemented as OTel attributes (not new SpanKind enum values, which would break OTLP). The taxonomy is derived from a coverage analysis: it is the minimum closed set under which all 14 fault classes in our benchmark have a span-level signal.

Adapter layer. Seven framework adapters use five qualitatively different binding strategies: callback (LangChain), life-cycle hook (CrewAI, AutoGen), span handler (LlamaIndex), monkey-patch (Anthropic SDK, OpenAI SDK), and manual context manager (custom). All adapters import their framework lazily at `instrument()` time — a hard requirement for a single observability library that ships in agent containers with diverse dependency closures.

Cross-boundary correlation. Three boundaries break OTel’s default propagation: asyncio task creation, process-pool execution, and inter-agent message routing. We solve each with a thin wrapper that serializes the W3C trace-context carrier and re-installs it on the receiver side.

Telemetry-driven circuit breaker. An OTel SpanProcessor that subscribes to the same stream as the exporter, maintains per-trace state (cost accumulator, reasoning-step counter, delegation graph, token-history), and fires configurable actions (log / callback / raise) when a policy matches. The breaker ships four default policies (reasoning loop, cost explosion, context overflow, delegation cycle) and accepts user-defined policies through a small Python DSL. This mechanism is impossible without the nine-kind vocabulary: a reasoning-loop detector cannot distinguish reasoning from delegation without per-kind attribution.

Evaluation

We report four measurement axes. Hardware: Apple M4 Pro, Python 3.12, opentelemetry-api/sdk 1.27.x. All raw data and reproduction commands ship with the artifact.

Per-span overhead. 10,000 iterations per kind (90,000 total): aggregate p50 11.7 μ s, p95 13.2 μ s, p99 28.2 μ s, mean 12.8 μ s. Throughput 58,771–66,396 spans/s depending on kind. Memory per span 3,103–3,191 bytes.

Scalability. 100-thread concurrent: 19,071 spans/s, p50 44.0 μ s. Long-running 1,200-span single trace: 7.6% latency

degradation between first and last decile. Disk-backed JSON exporter: 1,232% blocking overhead vs in-memory baseline (we surface this rather than hide it; the mitigation is the BatchSpanProcessor, which holds 5,335 spans/s end-to-end on the long trace).

Circuit breaker activation. Four-policy breaker installed on top of the per-span pipeline: +2.16 μ s p50 (+18.9%, 95% CI [+2.06, +2.25] across 5 independent runs); +3.31 μ s p99 (+22.9%); sub-millisecond absolute on operations whose wall-clock baseline is 100 ms–10 s.

Fault detection. Controlled fault-injection benchmark over 7 frameworks $\times$ 6 LLMs $\times$ 14 fault classes (3,780 runs). Aggregate FDR by condition: no telemetry 0.000; vanilla OTel, OTel GenAI semconv, OpenInference all tie at 0.429 (95% Wilson CI [0.389, 0.469]); AGENTSCOPE METADATA_ONLY and FULL both 0.612 (CI [0.572, 0.651]); AGENTSCOPE on a conformance-complete adapter 1.000 (CI [0.957, 1.000]). Difference between AGENTSCOPE and production baselines significant at $p < 0.001$. The per-fault matrix shows that the three baselines catch the same 6 bottom-up classes (tool failure, timeout, cost explosion, etc.) and miss the same 8 orchestration classes (circular delegation, reasoning loop, guardrail bypass, etc.); the 18.3-point absolute lift is the value

of the agent-specific schema, and the gap between 0.612 and 1.000 is a per-app conformance issue, not a metamodel-expressivity issue.

Real-LLM end-to-end. 45-run multi-agent topology study (hierarchical / parallel / sequential $\times$ 3 model back-ends $\times$ 5 seeds): 570 spans, zero orphan spans across boundaries (Clopper–Pearson 95% upper bound on true orphan rate: 0.52%), span-level cost attribution matches model provider billing within 1%.

Contribution

AGENTSCOPE is the first observability stack we know of to (a) treat agent-orchestration spans as enforceable runtime state rather than only diagnostic exhaust, and (b) report end-to-end overhead, scalability, and fault-detection-rate numbers for the GenAI tracing pipeline. The systems trade-offs (sync vs batched exporter, schema richness vs per-span memory, adapter fragility vs instrumentation coverage, breaker action choice) are reported in full. The full paper (under separate cover) provides per-class measurement, per-framework breakdown, threats-to-validity analysis, and reproducibility commands.